

ArenaBrawl

Naziv teme

ArenaBrawl – 2D igrice sa ciljem učenja i istraživanja Reinforcement Learning algoritama.

Definicija problema

Projektovanje agenata koji mogu autonomno donositi odluke u realnom vremenu unutar dinamičkih okruženja, jedan je od osnovnih problema savremene veštačke inteligencije. Postojeći sistemi veštačke inteligencije su ručno programirani da prate pravila – agent prati instrukcije, ali ne uči i ne prilagođava se. Takvi sistemi su krhki, funkcionišu samo u situacijama koje je programer predvideo i otkazuju čim se pojavi neočekivano ponašanje protivnika ili okruženja.

Reinforcement Learning uvodi drugačiji pristup – agent ne zna pravila, već ih otkriva sam kroz iteraciju sa okruženjem, nagradom i greškom, bez eksplicitnog programiranja ponašanja. Međutim, naivna primena Reinforcement Learning-a dovodi do overfittinga: agent nauči da pobjedi jednog konkretnog protivnika, ali ne razvija strategije koje se prenose na nepoznate situacije. Rešavanje ovog problema zahteva promišljenu strategiju treniranja koja agenta primorava da se prilagođava različitim protivnicima i formira opšte, prenosive strategije.

Za potrebe projekta, kreiraćemo igru, ona nije sam cilj projekta, već kontrolisano okruženje koje omogućava razvoj i testiranje algoritama. Sama igra je jednostavna i lako razumljiva, detalji su opisani u tekstu koji sledi.

Cilj projekta je implementacija, treniranje i poređenje četiri algoritma Reinforcement Learning-a: DQN, Dueling DQN, PPO i A2C – na zajedničkom okruženju i pod identičnim uslovima. Sami algoritmi razlikuju po načinu učenja: *value-based* i *policy-based*.

Projekat utvrđuje razlike u brzini konvergencije, stabilnosti treniranja i kvalitet naučenih strategija.

Fictitious Self-Play strategija treniranja osigurava da agenti ne overfituju na jednog protivnika, već razvijaju strategije primenljive i na nepoznatog protivnika – uključujući čoveka.

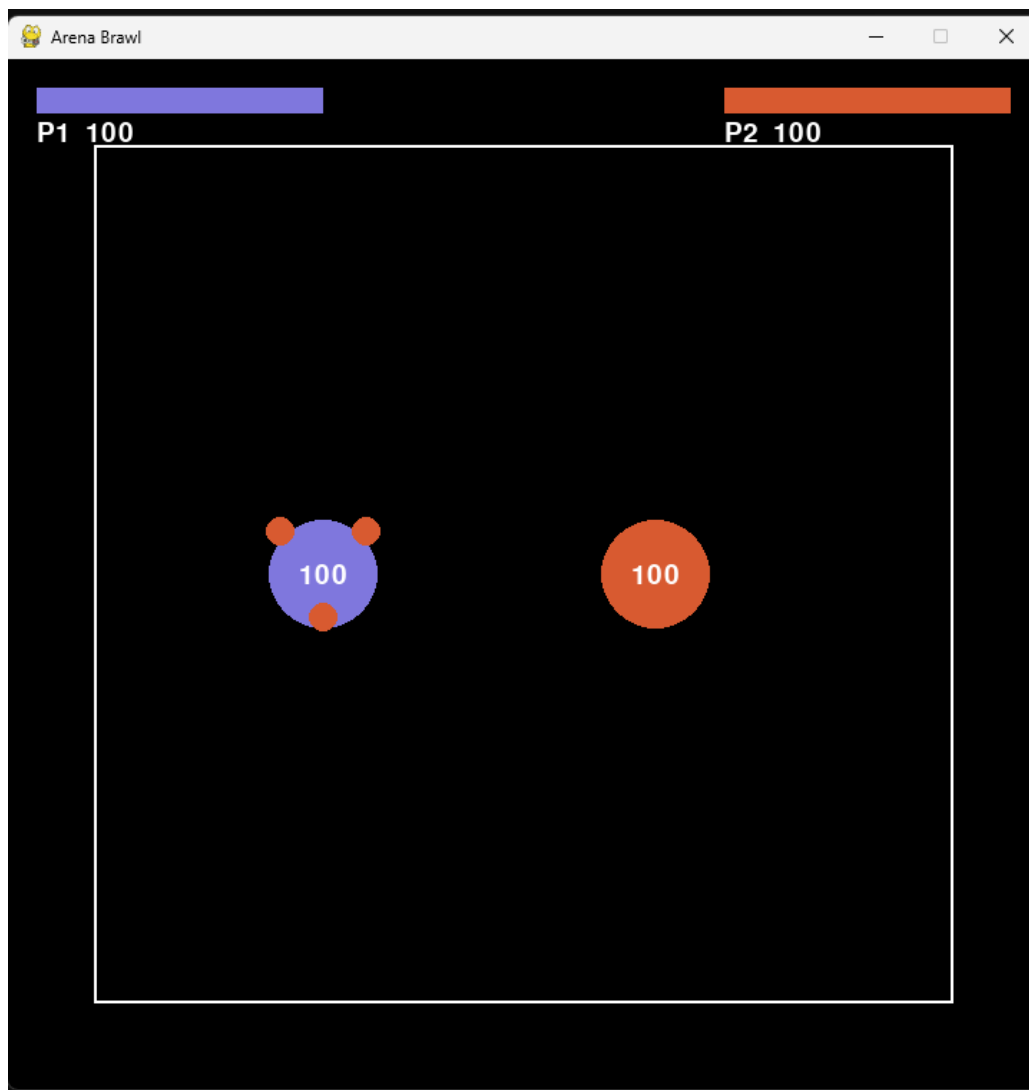
Skup podataka

Projekat ne koristi unapred postojeći skup podataka. Umesto toga, podaci se generišu tokom igranja.

Okruženje igre

Okruženje je 2D borbena igra implemenirana u Pygame-u umotana u Gymnasium interfejs. Dva karaktera se bore unutar ograničene arene i svaki poseduje jedinstvenu sposobnost. Epizoda počinje postavljanjem oba lika na početne pozicije sa punim životnim poenima i završava se kada jedan od karaktera izgubi sve životne poene (za potrebe učenja uvedeno je i vremensko ograničenje).

Prostor akcija sadrži 10 odvojenih akcija: kretanje u 8 pravaca (gore-dole, levo-desno, dijagonale), mirovanje i upotreba sposobnosti.



Vektor stanja

U svakom koraku agent prima vektor od 12 normalizovanih vrednosti:

1. Pozicija agenta (player_x, player_y)
2. Pozicija protivnika (opponent_x, opponent_y)
3. Životni poeni agenta (player_hp)
4. Životni poeni protivnika (opponent_hp)
5. *Cooldown*
6. Distanca između karaktera
7. Brzina kretanja protivnika (opponent_vx, opponent_vy)
8. Pozicija najbližeg projektila (projectile_x, projectile_y)

Ciljno obeležje je **reward** – vrednost koju agent maksimizuje tokom treniranja. Kodira nanесenu štetu, primljenu štetu, ishod epizode, vremensku kaznu.

Format iskustva

U svakom koraku igre agent generiše jednu torku iskustva: (**state, action, reward, done**)

Memorijski bafer

DQN i Dueling DQN koriste *replay buffer* – memorija koja čuva iskustvo prethodnih epizoda. Svaki put kad uči, agent nasumično izvlači iskustva iz te memorije, što ga sprečava da uči samo iz onoga što se nedavno desilo.

PPO i A2C koriste *rollout buffer* – privremena memorija koja čuva iskustva isključivo trenutne epizode i odbacuje ih nakon ažuriranja.

Veličine bafera i trajanje treniranja se određuju empirijski tokom implementacije.

Metodologija

Projekat implementira i upoređuje četiri algoritma Reinforcement Learning-a:

- 1) DQN (Deep Q-Network)
- 2) Dueling DQN
- 3) PPO (Proximal Policy Optimization)
- 4) A2C (Advantage Actor-Critic)

Strategija treniranja (Fictitious Self-Play)

Agent trenira protiv istorijske raspodele svojih prethodnih verzija. Svaki agent periodično čuva trenutne težine i biase. Protivnici za treniranje se biraju iz rastuće zbirke tih snimaka, pri čemu noviji snimci imaju veću težinu ali stari ostaju prisutni radi raznovrsnosti strategije.

Treniranje se odvija u četiri faze:

- 1) Agent trenira isključivo protiv lakog skriptovanog bota koji samo prati protivnika i napada u dometu. Zbirka snimaka još ne postoji. Agent uči osnove: kretanje, upotreba sposobnosti, elementarne akcije.
- 2) Zbirka se proširuje sa prvim iteracijama. Agent trenira pretežno protiv snimaka, uz manji deo skriptovanog bota – bot dodaje kiting mehaniku (hit&run) i bočno kretanje.
- 3) Zbirka raste. Snimci postaju primarni protivnik. Bot dodaje povlačenje pri niskim životnim poenima i pažljiviju upotrebu sposobnosti
- 4) Bot ostaje prisutan u malom procentu mečeva, obezbeđuje sigurnost – agent neće doživeti katastrofalno zaboravljanje

Svaki karakter po završetku treniranja čuva parametre u .pth datoteku. Datoteka sadrži isključivo naučene težine, odnosno sve znanje akumulirano tokom treniranja. Pri pokretanju igre, agent učitava odgovarajuću datoteku i odmah igra na naučenom nivou bez ponovnog treniranja.

Način evaluacije

Tokom treniranja:

- *Episode reward curve* – ukupna nagrada po epizodi (odigranom meču), očekivano je da raste kako agent napreduje.
- *Rolling win rate* – procenat pobeda u poslednji 100 odigranih epizoda, očekuje se da vremenom raste i govori u kojoj meri agent pobeđuje, takođe je i najlakše za ispratiti.
- *Epsilon decay curve* – Epsilon prikazuje koliko se agent nasumično ponaša. Vremenom se epsilon smanjuje, dok se agent oslanja na ono što je naučeno.
- *Policy entropy* – ekvivalent epsilon krive za PPO i A2C algoritme. Visoka entropija znači raznovrsnije akcije (eksploracija), niska entropija znači opredeljenost za konkretne strategije (eksploatacija).
- *Loss curve* – govori koliko su pogrešne bile predikcije tokom vremena. Na početku će vrednost biti visoka jer mreža ne zna ništa. Pad označava da se agent što bolje snalazi.
- *Episode length* – broji koliko koraka je trajala svaka epizoda. Kraće borbe znače da agent postaje odlučniji. Ako bi epizode trajale duže, to može biti indikacija da agent uči da beži umesto da se bori.

Nakon treniranja – kružni turnir:

- Svaki istrenirani lik se bori protiv svakog drugog lika. Rezultati se formiraju u matrici 10x10 (ili koliko već karaktera bude bilo NxN), gde svako polje predstavlja *win rate* protiv određenog protivnika.

Ablation study (analiza doprinosa komponenti):

- Jedan agent se trenira bez Fictitious Self-Play – isključivo protiv skriptovanog bota tokom celog treniranja. Rezultati se porede sa treniranjem sa FSP-om. Ovako prikazujemo značaj strategije treniranja i opravdavamo njenu upotrebu.

Finalni test – Human vs AI:

- Finalni test je borba čoveka (nas) protiv naučenih agenata. Win rate agenta u ovim mečevima meri sposobnost generalizacije van trenažnih uslova. Agent koji pobeđuje samo AI protivnike ali gubi od čoveka overfitovao je na trenažne uslove i nije razvio prave strategije.

Tehnologije

- Python 3.12 – primarni programski jezik
- Pygame – renderovanje, Human vs AI
- Gymnasium – RL interfejs – logika je umotana u Gymnasium interfejs što omogućava treniranje bez renderovanja i brže učenje
- PyTorch – Implementacija neuronskih mreža
- Wandb (weights and biases) – Praćenje treniranja u realnom vremenu
- Pandas – Analiza borbi
- Matplotlib – vizualizacija rezultata
- Python csv modul – Beleženje svakog meča u trajnu evidenciju

Primeri

- [OpenAI Five](#) – agenti za igru Dota 2 trenirani sa PPO algoritmom
- [AlphaGo/AlphaZero](#) – DeepMind-ovi agenti za Go i šah korišćenjem MCTS
- [OpenAI Gym](#) – Standardna referentna okruženja za istraživanje RL, uključujući Atari igre

Relevantna literatura

- [PyTorch Official DQN Tutorial](#)
- [Gymnasium Documentation](#)
- [Spinning Up in Deep RL – OpenAI](#)
- [TwoMinutePapers](#)
- [Andrej Karpathy](#)
- [3Blue1Brown neural network series](#)